

Optimización de un rango de push/fold en póker heads-up con un Algoritmo Genético

Desafío Práctico — Algoritmos Evolutivos I (2026)

Maestría en Inteligencia Artificial — Facultad de Ingeniería, Universidad de Buenos Aires

Autor: Facundo Rivas · Técnica: Algoritmos Genéticos (GA)

Repositorio con el código fuente: <https://github.com/Nestiii/AE1>

Resumen

Se aplica un Algoritmo Genético a un problema real de teoría de juegos del póker: hallar la estrategia óptima de *push/fold* (ir all-in o retirarse) en Texas Hold'em *heads-up* con stacks cortos. La estrategia se codifica como un cromosoma binario de 169 bits —uno por cada mano inicial distinta— y se evalúa con un modelo de valor esperado construido sobre *equities* reales de póker. El GA converge de forma estable al óptimo, que en esta formulación se conoce analíticamente y se usa para validar el resultado: la solución hallada coincide con él en las 169 manos.

1. El problema

En Texas Hold'em *heads-up* (un jugador contra otro) con stacks cortos, la teoría de juegos muestra que la estrategia óptima antes del flop se reduce a una decisión binaria del botón / ciega chica (SB): **ir all-in (push) o retirarse (fold)**. Jugar «push o fold» evita las decisiones posteriores (flop, turn, river), que con stacks cortos aportan poco valor y mucho riesgo, y por eso es la forma en que se juega y se estudia esta etapa del torneo.

El jugador debe decidir, para **cada una de las 169 manos iniciales** estratégicamente distintas (13 pares, 78 *suited* y 78 *offsuit*), si la pushea o la foldea. Esa decisión conjunta es un *rango de push*, y existen $2^{169} \approx 7,5 \cdot 10^{50}$ rangos posibles: un espacio de búsqueda enorme, imposible de recorrer por fuerza bruta e ideal para una metaheurística como un Algoritmo Genético.

La elección del GA es natural porque el rango se representa directamente como un **vector binario de 169 bits** (1 = push, 0 = fold), que es exactamente el cromosoma de un GA binario clásico.

2. Modelo del juego y función de fitness

Con un stack efectivo de S *big blinds* (BB), ciega chica 0,5 y ciega grande 1, el valor esperado (EV) de la SB en cada resultado es:

Acción de la SB	Resultado	EV (BB)
Fold	pierde su ciega chica	-0,5
Push y la BB foldea	se lleva la ciega grande	+1,0
Push y la BB paga	showdown all-in	$S \cdot (2 \cdot eq - 1)$

donde eq es la *equity* (probabilidad de ganar el showdown) del héroe contra la mano del villano. Para cada mano h del héroe, promediando sobre el rango con el que la BB paga:

$$EV_{\text{push}}(h) = \sum_j w_j \cdot [c_j \cdot S \cdot (2 \cdot eq_{hj} - 1) + (1 - c_j) \cdot 1] \quad EV_{\text{fold}} = -0,5$$

donde j recorre las 169 clases del villano, w_j es la probabilidad a priori de esa clase y $c_j \in \{0,1\}$ indica si la BB paga con ella. El *fitness* de un cromosoma x (ganancia esperada por mano) es:

$$\text{fitness}(x) = \sum_h w_h \cdot [x_h \cdot \text{EV_push}(h) + (1 - x_h) \cdot \text{EV_fold}]$$

El siguiente fragmento (de `src/pushfold/game.py`) vectoriza el cálculo de `EV_push` contra el rango fijo de la BB:

```
def _compute_ev_push(self):
    S = self.stack_bb
    w = self.weights # prob. a priori de cada mano
    call = self.bb_call_mask.astype(float) # 1 si la BB paga con esa mano
    fold = 1.0 - call
    showdown = S * (2.0 * self.equity - 1.0) # EV all-in héroe vs villano
    ev_when_called = (showdown * (w * call)).sum(axis=1)
    ev_when_folded = 1.0 * float((w * fold).sum()) # gana la ciega grande
    return ev_when_called + ev_when_folded
```

Como el rango de la BB es fijo, `EV_push(h)` no depende de las demás manos: el problema es **separable** y su óptimo se conoce en forma cerrada —pushear si y solo si `EV_push(h) > EV_fold`—. Ese óptimo analítico se usa como verdad de referencia para validar la convergencia del GA.

3. El Algoritmo Genético

Se implementó un GA binario clásico con los operadores estándar. Cada individuo es un vector de 169 bits y la población, una matriz (tamaño_población × 169).

Selección por torneo (tamaño 3): se eligen 3 individuos al azar y gana el de mayor fitness.

Cruza uniforme: cada gen del hijo se hereda de uno u otro padre. **Mutación bit-flip**: cada gen se invierte con probabilidad $\approx 1/169$ (≈ 1 bit esperado por cromosoma). **Elitismo**: los 2 mejores pasan intactos, lo que garantiza que el mejor fitness nunca empeore (curva de convergencia monótona).

Núcleo del bucle evolutivo (`src/pushfold/ga.py`):

```
for gen in range(config.n_generations):
    new_pop = np.empty_like(pop)
    elite = np.argsort(-fitness)[:config.elitism] # elitismo
    new_pop[config.elitism:] = pop[elite]
    for i in range(config.elitism, config.pop_size):
        p1 = _tournament_select(pop, fitness, k, rng)
        p2 = _tournament_select(pop, fitness, k, rng)
        child = _uniform_crossover(p1, p2, cx_rate, rng)
        new_pop[i] = _mutate(child, mut_rate, rng) # bit-flip
    pop = new_pop
    fitness = fitness_fn(pop) # fitness vectorizado
```

Parámetro	Valor
Tamaño de población	80
Generaciones	120
Prob. de cruce	0,9
Prob. de mutación	1/169 por gen
Torneo / Elitismo	3 / 2

4. Metodología experimental

La pieza de datos central es una **matriz 169×169 de equity all-in preflop**: `equity[i][j]` es la probabilidad de que la mano *i* gane el showdown contra la *j* con las 5 cartas comunitarias repartidas al azar. Se estima por Monte Carlo con el evaluador de manos en `C eval7` (25.000 repartos por matchup), respetando el *card removal*, y se precomputa una sola vez (≈ 25 s) guardándola en el repositorio. Como el GA y el óptimo analítico se calculan sobre esa misma matriz, la validación es exacta con independencia del pequeño ruido de estimación.

Los experimentos usan un stack efectivo de 10 BB y una BB que paga con el 50% de las manos más fuertes (rival fijo). Para medir robustez se corre el GA con 30 semillas independientes.

5. Resultados

El GA converge de forma estable al óptimo analítico: coincide con él en las **169/169 manos** y alcanza un fitness de 0,0149 BB/mano, pusheando el 43% de las manos. La Figura 1 muestra la curva de convergencia (entregable obligatorio): el mejor y el promedio de la población suben hasta estabilizarse en el óptimo.

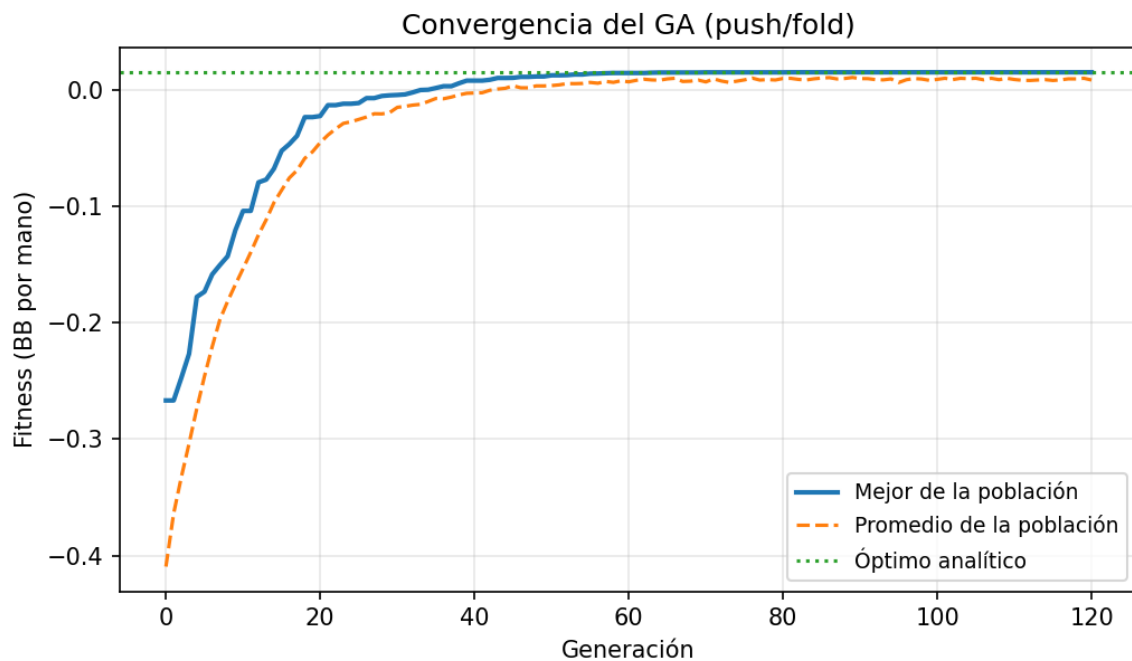


Figura 1. Convergencia del GA: mejor y promedio de la población por generación, con el óptimo analítico de referencia.

Como el problema es separable, todas las corridas terminan alcanzando el óptimo. Para que el diagrama de caja sea informativo, la Figura 2 muestra el fitness final a distintos presupuestos de generaciones (15/30/60/120) sobre las 30 corridas: al aumentar el presupuesto, la mediana sube hacia el óptimo y la dispersión entre corridas se reduce, evidenciando la fiabilidad del método.

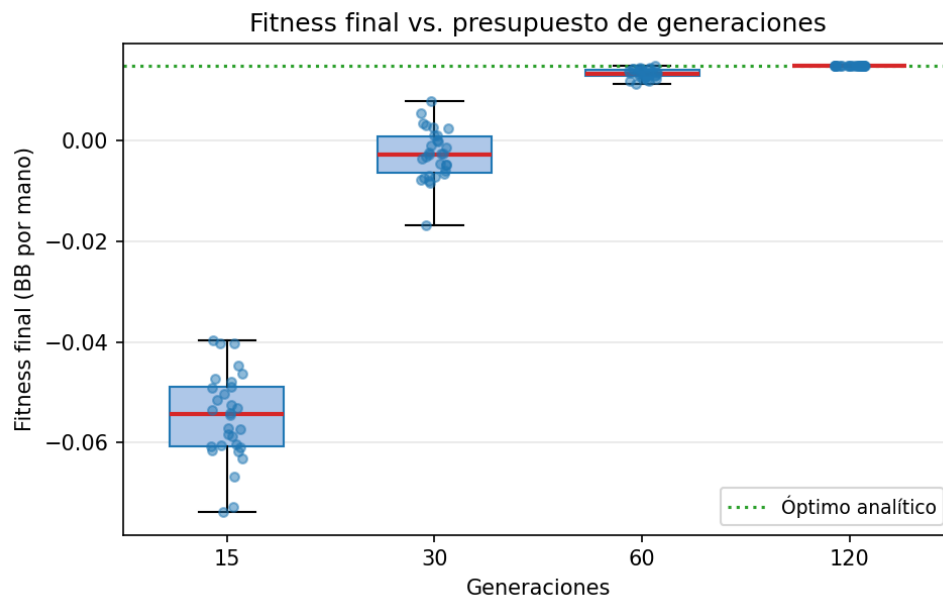


Figura 2. Diagrama de caja del fitness final vs. presupuesto de generaciones (30 corridas independientes).

Finalmente, la Figura 3 presenta el cromosoma ganador como la clásica grilla 13×13 de manos (verde = push, gris = fold). El rango obtenido tiene pleno sentido pokerístico: se pushean todos los pares, cualquier mano con un as o un rey (incluso K2), las manos altas (*broadways*) y algunos conectores *suiteds*. Que manos débiles como K2 entren se explica por el modelo de EV: con la BB foldeando casi la mitad de las veces y la ciega chica ya posteadada, empujar pierde menos que rendirse. No hay ninguna celda marcada en rojo, es decir, no hay diferencias con el óptimo analítico.

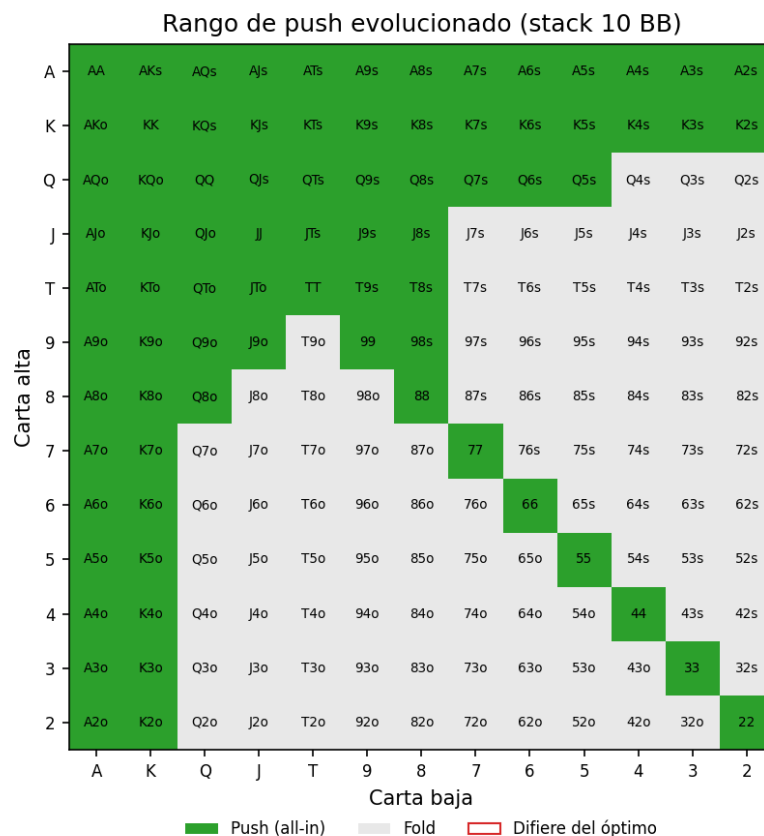


Figura 3. Rango de push evolucionado (stack 10 BB). Verde = push, gris = fold; borde rojo marcaría diferencias con el óptimo (ninguna).

6. Inconvenientes encontrados y cómo se resolvieron

a) Costo de calcular las equities. Estimar por Monte Carlo en Python puro las equities de las 169×169 combinaciones era demasiado lento (no terminaba en minutos). Se resolvió delegando la evaluación de manos al evaluador en C `eval7` y, sobre todo, **precomputando la matriz una sola vez** y versionándola en el repositorio; así el GA corre después solo con NumPy, en segundos.

b) Advertencias espurias de `matmul`. En macOS con Apple Accelerate y NumPy 2.0, el producto matriz-vector emitía *warnings* de *divide-by-zero/overflow* pese a devolver el resultado correcto (se verificó comparando contra el producto calculado a mano). Se reemplazó el `matmul` por una reducción *elementwise* equivalente, que no dispara el problema en ningún backend.

c) Diagrama de caja degenerado. Al ser el problema separable, las 30 corridas alcanzaban exactamente el mismo óptimo y el box plot colapsaba a un punto, sin información. Se replanteó para mostrar el fitness final a distintos presupuestos de generaciones, que sí exhibe dispersión y comunica la velocidad de convergencia (Figura 2).

7. Conclusiones y trabajo futuro

El Algoritmo Genético resolvió con éxito el problema de *push/fold*: codificando la estrategia como 169 bits y evaluándola con un modelo de EV sobre equities reales, converge de forma estable y reproducible al óptimo, validado contra su solución analítica (0/169 manos de diferencia). El resultado no solo es correcto numéricamente sino interpretable: reproduce un rango de all-in con sentido para cualquier jugador de póker.

La principal limitación es que el rival (rango de call de la BB) es fijo, lo que vuelve el problema separable. La extensión natural es **coevolucionar** los rangos de push de la SB y de call de la BB como un juego de suma cero, para aproximar el **equilibrio de Nash** de push/fold; allí el problema deja de ser separable y el Algoritmo Genético cobra un rol más rico como buscador de equilibrios. El código completo, la notebook reproducible en Google Colab y las instrucciones están en el repositorio enlazado más abajo.

Repositorio (código fuente)	https://github.com/Nestiii/AE1
Notebook reproducible (Colab)	https://colab.research.google.com/drive/1y-XuN03sDg1YcVJ02KtwqX7zpu5MHd1E?usp=sharing